

WEB API & ANGULAR CON VISUAL STUDIO .NET CORE

Capítulo 5: Controladores





OBJETIVOS

- Configurar rutas en asp.net web api.
- Diferenciar entre los diferentes tipos de actions.
- Crear formatos personalizados en asp net web api.





AGENDA

1. Rutas
2. Action Results y formateo de resultado
3. Formateador de respuesta personalizado





1. RUTAS

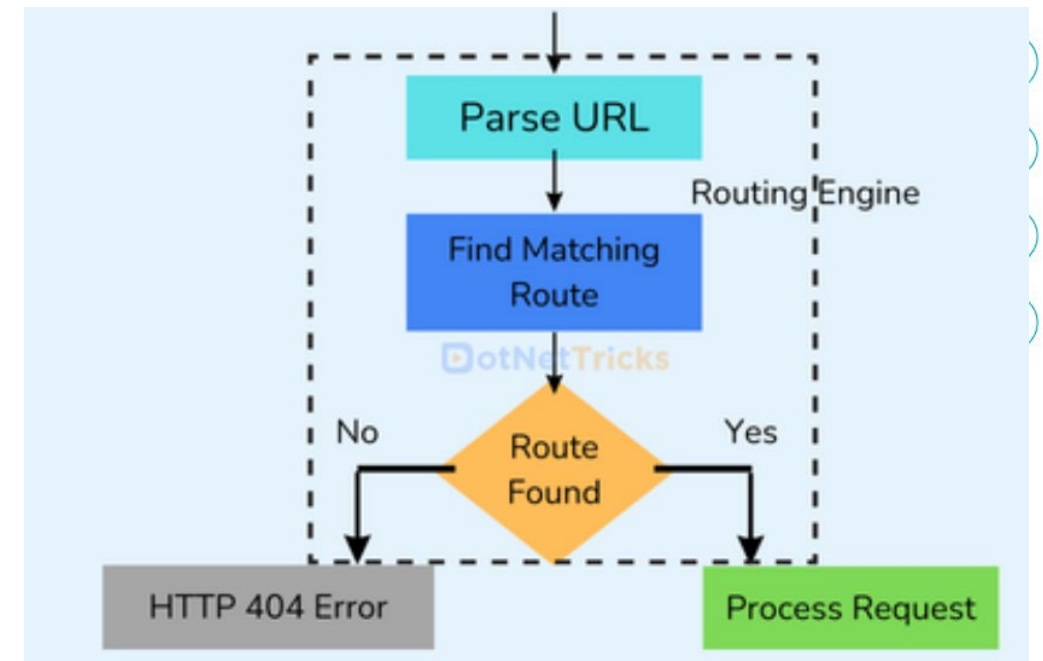
Asp Net Web API recibe una solicitud HTTP, intenta hacer coincidir el URI con una de las plantillas de ruta de la tabla de enrutamiento. Si no coincide ninguna ruta, el cliente recibe un error 404.

Por ejemplo, los siguientes URI coinciden con la ruta predeterminada:

/api/contacts
/api/contacts/1
/api/products/gizmo1

Sin embargo, el siguiente URI no coincide, ya que carece del segmento "api":

/contacts/1





1. RUTAS

Las rutas se definen de manera global o a nivel de los controladores.

```
routes.MapHttpRoute(  
    name: "ActionApi",  
    routeTemplate: "api/{controller}/{id}/{action}",  
    defaults: new { action = "DirectoryInfo", id = RouteParameter.Optional }  
);
```

Global



```
[Route("api/[controller]")]  
[ApiController]
```

1 reference

```
public class AlumnoController : ControllerBase
```

En los controladores



1. RUTAS

Ejemplo

Definiendo el template de rutas

```
routes.MapHttpRoute(  
    name: "API Default",  
    routeTemplate: "api/{controller}/{id}",  
    defaults: new { id = RouteParameter.Optional }  
);
```

Definiendo los métodos (actions) del controlador

```
public class ProductsController : ControllerBase  
{  
    public IEnumerable<Product> GetAllProducts() { }  
    public Product GetProductById(int id) { }  
    public HttpResponseMessage DeleteProduct(int id) { }  
}
```

Match entre los métodos en el controlador y el template de rutas

Verbo HTTP	Ruta de acceso de URI	Acción	Parámetro
GET	api/products	GetAllProducts	(none)
GET	api/products/4	GetProductById	4
Delete	api/products/4	DeleteProduct	4
POST	api/products	(sin coincidencia)	





1. RUTAS

Usando los verbos http para definir el nombre de la ruta.

- [HttpGet]
- [HttpPut]
- [HttpPost]
- [HttpDelete]
- [HttpHead]
- [HttpOptions]
- [HttpPatch]

```
[HttpPost("Registrar")]
0 references
public async Task<IActionResult> Registrar(RegistrarAlumnoRequest request)
{
    var response = await _mediator.Send(request);

    return Ok(response);
}
```





1. RUTAS

Atributo AcceptVerbs

Puede usar el atributo [AcceptVerbs] para especificar una lista de verbos

```
public class ProductsController : ControllerBase
{
    [AcceptVerbs("GET", "HEAD")]
    public Product FindProduct(id) { }

    // WebDAV method
    [AcceptVerbs("MKCOL")]
    public void MakeCollection() { }
}
```

Atributo NonAction

Para evitar que un método se invoque como una acción

```
// Not an action method.
[NonAction]
public string GetPrivateData() { ... }
```





2. CONTROLADORES

Una API web basada en controlador consta de una o varias clases de controlador que se derivan de **ControllerBase**. La plantilla de proyecto de API web proporciona un controlador de inicio:

```
[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
```

Métodos disponibles en el ControllerBase

Método	Notas
BadRequest	Devuelve el código de estado 400.
NotFound	Devuelve el código de estado 404.
PhysicalFile	Devuelve un archivo.
TryUpdateModelAsync	Invoca el enlace de modelo .
TryValidateModel	Invoca la validación de modelos .





2. CONTROLADORES

Atributos

Se pueden usar para configurar el comportamiento de los controladores API web y los métodos de acción.

Atributo	Descripción
HttpPost	Vervo http admitido en la acción del controlador
ProducesResponseType	Configura el código de retorno del estado de HTTP
Route	Especifica el patrón de dirección URL de un controlador o una acción.
Bind	Especifica el prefijo y las propiedades que se incluirán en el enlace de modelo.
HttpGet	Vervo http admitido en la acción del controlador
Consumes	Especifica los tipos de datos que acepta una acción
Produces	Especifica los tipos de datos que devuelve una acción

```
[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
```

```
[HttpPost]
[ProducesResponseType(StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public ActionResult<Pet> Create(Pet pet)
```

```
[Produces(MediaTypeNames.Application.Json)]
[Route("[controller]")]
```



2. CONTROLADORES

Atributo origen de enlace

Un atributo de origen de enlace define la ubicación del valor del parámetro de una acción.

Atributo	Origen de enlace
[FromBody]	Cuerpo de la solicitud
[FromForm]	Datos del formulario en el cuerpo de la solicitud
[FromHeader]	Encabezado de la solicitud
[FromQuery]	Parámetro de la cadena de consulta de la solicitud
[FromRoute]	Datos de ruta de la solicitud actual
[FromServices]	Servicio de solicitud insertado como parámetro de acción
[AsParameters]	Parámetros de métodos

```
[HttpGet]  
public ActionResult<List<Product>> Get(  
    [FromQuery] bool discontinuedOnly = false)  
{  
    ...  
}
```





3. ACTION RESULTS Y FORMATEO DE RESULTADO

ASP.NET Core proporciona las siguientes opciones para los tipos de valor devueltos de acción del controlador de API web:

1. Tipo específico:

La acción más básica devuelve un tipo de dato primitivo o complejo, por ejemplo, string o un objeto personalizado.

```
[HttpGet]
```

```
public Task<List<Product>> Get() =>
```

```
    _productContext.Products.OrderBy(p => p.Name).ToListAsync();
```





3. ACTION RESULTS Y FORMATEO DE RESULTADO

Devolver IEnumerable<T> o IEnumerableAsync<T>

ASP.NET Core almacena en búfer el resultado de las acciones que devuelven IEnumerable<T> antes de escribirlas en la respuesta. Considere la posibilidad de declarar el tipo de valor devuelto de la signatura de la acción como IEnumerableAsync<T> para garantizar la iteración asíncrona.

```
[HttpGet("syncsale")]
public IEnumerable<Product> GetOnSaleProducts()
{
    var products = _productContext.Products.OrderBy(p => p.Name).ToList();

    foreach (var product in products)
    {
        if (product.IsOnSale)
        {
            yield return product;
        }
    }
}
```

```
[HttpGet("asyncsale")]
public async IEnumerableAsync<Product> GetOnSaleProductsAsync()
{
    var products = _productContext.Products.OrderBy(p => p.Name).AsAsyncEnumerable();

    await foreach (var product in products)
    {
        if (product.IsOnSale)
        {
            yield return product;
        }
    }
}
```





3. ACTION RESULTS Y FORMATEO DE RESULTADO

2. IActionResult

El tipo de valor devuelto IActionResult resulta adecuado cuando existen varios tipos de valor devuelto ActionResult posibles en una acción. Los tipos ActionResult representan varios códigos de estado HTTP. Cualquier clase no abstracta derivada de ActionResult se considera un tipo de valor devuelto válido. Algunos tipos de valor devueltos comunes en esta categoría son BadRequestResult (400), NotFoundResult (404) y OkObjectResult (200).



```
public IActionResult GetById_IActionResult(int id)
{
    var product = _productContext.Products.Find(id);
    return product == null ? NotFound() : Ok(product);
}
```

```
public async Task<IActionResult> CreateAsync_IActionResult(Product product)
{
    if (product.Description.Contains("XYZ Widget"))
    {
        return BadRequest();
    }

    _productContext.Products.Add(product);
    await _productContext.SaveChangesAsync();

    return CreatedAtAction(nameof(CreateAsync_IActionResult), new { id = product.Id }, product);
}
```




3. ACTION RESULTS Y FORMATEO DE RESULTADO

3. ActionResult<T>

ASP.NET Core incluye el tipo de valor devuelto ActionResult<T> para las acciones del controlador de la API web. Permite devolver un tipo derivado de ActionResult o devolver un tipo específico.

```
public ActionResult<Product> GetById_ActionResultOfT(int id)
{
    var product = _productContext.Products.Find(id);
    return product == null ? NotFound() : product;
}
```

```
public async Task<ActionResult<Product>> CreateAsync_ActionResultOfT(Product product)
{
    if (product.Description.Contains("XYZ Widget"))
    {
        return BadRequest();
    }

    _productContext.Products.Add(product);
    await _productContext.SaveChangesAsync();

    return CreatedAtAction(nameof(CreateAsync_ActionResultOfT), new { id = product.Id }, product);
}
```





3. ACTION RESULTS Y FORMATEO DE RESULTADO

4. HttpResults

Además de los tipos de resultados incorporados específicos de MVC (`ActionResult` y `ActionResult<T>`), ASP.NET Core incluye los tipos `HttpResults` que pueden usarse tanto en API mínimas como en API Web.

El espacio de nombres `Microsoft.AspNetCore.Http.HttpResults` contiene clases que implementan la interfaz `IResult`. La interfaz `IResult` define un contrato que representa el resultado de un punto de conexión HTTP. La clase estática `Results` se usa para crear distintos objetos `IResult` que representan diferentes tipos de respuestas.

```
public IResult GetById(int id)
{
    var product = _productContext.Products.Find(id);
    return product == null ? Results.NotFound() : Results.Ok(product);
}
```



4. FORMATEADOR DE RESPUESTA PERSONALIZADO

ASP.NET Core MVC admite el intercambio de datos en las API Web con formateadores de entrada y salida. El enlace de modelos usa formateadores de entrada. Los formateadores de salida se usan para dar formato a las respuestas.

El marco proporciona formateadores de entrada y salida integrados para JS ON y XML. Proporciona un formateador de salida integrado para texto sin formato, pero no proporciona un formateador de entrada para texto sin formato.





4. FORMATEADOR DE RESPUESTA PERSONALIZADO

Cómo crear un formateador personalizado

- Para serializar los datos enviados al cliente, cree una clase de formateador de salida.
- Para deserializar los datos recibidos del cliente, cree una clase de formateador de entrada.
- Añade instancias de clases de formateadores a las colecciones InputFormatters y OutputFormatters en MvcOptions.

Para crear un formateador:

- Derive la clase de la clase base adecuada.
- La aplicación de ejemplo se deriva de TextOutputFormatter y TextInputFormatter.
- Especifique en el constructor los tipos de medios y codificaciones admitidos.
- Invalide los métodos CanReadType y CanWriteType.
- Invalide los métodos ReadRequestBodyAsync y WriteResponseBodyAsync.

```
public class VcardOutputFormatter : TextOutputFormatter
{
    public VcardOutputFormatter()
    {
        SupportedMediaTypes.Add(MediaTypeHeaderValue.Parse("text/vcard"));

        SupportedEncodings.Add(Encoding.UTF8);
        SupportedEncodings.Add(Encoding.Unicode);
    }

    protected override bool CanWriteType(Type? type)
        => typeof(Contact).IsAssignableFrom(type)
           || typeof(IEnumerable<Contact>).IsAssignableFrom(type);
}
```



4. FORMATEADOR DE RESPUESTA PERSONALIZADO

```
public class VcardOutputFormatter : TextOutputFormatter
```

```
public VcardOutputFormatter()
{
    SupportedMediaTypes.Add(MediaTypeHeaderValue.Parse("text/vcard"));

    SupportedEncodings.Add(Encoding.UTF8);
    SupportedEncodings.Add(Encoding.Unicode);
}
```

```
protected override bool CanWriteType(Type? type)
    => typeof(Contact).IsAssignableFrom(type)
        || typeof(IEnumerable<Contact>).IsAssignableFrom(type);
```





LABORATORIO N° 1: DEFINIR LA RUTA PARA EL ENDPOINT DE REGISTRO Y CONSULTA DE ALUMNOS

En este laboratorio, podrás conocer cómo funciona la configuración de rutas.





LABORATORIO N° 2: PARA EL REGISTRO DE ALUMNOS, ADECUAR LOS ACTION RESULT PARA RETORNAR HTTP 200, 400 Y 500

En este laboratorio, podrás definir retornos en un endpoint y usar acciones asíncronas.





TAREA N° 1: CREAR UN CONTROLADOR PARA REGISTRAR LA VENTA DE CURSOS

Al finalizar la tarea, lograrás:

- Registrar de la venta: Usando el método post para el registro.
- Consultar la venta por el código de venta: Usando el método get y el identificador de la venta deber ser parte de la URL
- Tener el listado de ventas y consultar por rango de fechas: Usando el método get.





LECTURAS ADICIONALES

Para obtener información adicional, puede consultar:

- Microsoft Build. (30 de noviembre de 2023). *Creación de API web con ASP.NET Core*. <https://learn.microsoft.com/es-mx/aspnet/core/web-api/?view=aspnetcore-8.0&viewFallbackFrom=aspnetcore-8.0%E2%80%8B>
- Microsoft Build. (30 de noviembre de 2023). Tipos de valor devuelto de acción del controlador de la API web de ASP.NET Core. <https://learn.microsoft.com/es-mx/aspnet/core/web-api/action-return-types?view=aspnetcore-8.0>





RESUMEN

- Para la configuración de las rutas en asp.net web api se definen de manera global o a nivel de los controladores
- Hay diferentes tipos de actions, y cada uno tiene una utilidad acorde a las necesidades.
- Para crear los formatos personalizados se debe seguir una secuencia en asp net web api.



